AI / ML / Agent Ideas for an O+O R_{AA} Thesis

with sPHENIX EMCal Calibration

How to use this document

Each idea is tagged with **Tier** (priority for the thesis), **Effort**, **Stack** (what to actually install / call), and **First test** (a concrete thing to try in <1 day to prove out the approach). Tier 1 items earn a thesis chapter or section. Tier 2 items make you faster and produce nicer outputs but don't get cited. Tier 3 items are exploratory — only chase them if you finish the others or fall in love.

The honest picks for you: Tier 1 idea #1 (cluster energy regression) and Tier 2 idea #4 (RAG over sPHENIX docs) and #5 (ROOT macro agent). Those three give the best return on weeks-of-effort.

1 Tier 1 — ML inside the measurement (thesis-critical)

1. Cluster-level photon energy regression

What: A BDT or shallow NN that takes EMCal cluster shower-shape variables and predicts the photon energy correction on top of your standard tower calibration. Inputs: σ_η, σ_ϕ, dispersion, $E_{\max}/E_{\text{total}}$, n_{towers}, position-in-tower (η_{local}, ϕ_{local}), cluster η. Target: $E_{\text{true}}/E_{\text{reco}}$ from single-photon MC.
Why thesis: Tightens your π^0 mass peak, which is the dominant contributor to your photon-energy-scale systematic on R_{AA}. It is a clean, self-contained chapter: "standard calibration → ML refinement → closure on data-driven π^0." CMS H→ $\gamma\gamma$ has been doing this since Run 1, so the validation playbook is well-trodden.
Stack: `xgboost` or `lightgbm` (BDT, fast, no GPU); `PyTorch` if you want a NN. Convert ROOT trees with `uproot` + `awkward`. Inference back in C++ via `TMVA::Experimental::RBDT` or ONNX Runtime.
First test (1 day): Pull single-photon MC, dump 10 shower-shape branches to a parquet file with `uproot`, train an `xgboost` regressor in a Jupyter notebook, plot $E_{\text{true}}/E_{\text{reco}}$ before vs. after correction in pT bins. If you see the resolution improve in even one p_T bin, the path is real.
Effort: 6–10 weeks for a publishable-quality version with full systematics.

2. Photon vs. merged-π^0 classifier (high-p_T)

What: BDT/NN classifier on the same shower-shape inputs, trained to separate single photons from merged-π^0 clusters. Lets you push your R_{AA} measurement higher in p_T than cuts alone allow.
Why thesis: sPHENIX EMCal granularity makes π^0 photons start merging around $p_T \sim 12$–15 GeV. Your O+O suppression story may or may not need that high-p_T reach — if the physics motivation is "does suppression onset look like p+Au or like Au+Au", then yes, the high-p_T tail matters.
Stack: Same as above. Add `shap` for feature importance — referees love it.
First test (1 day): Train on single-γ vs. single-π^0 MC with $p_T > 10$ GeV. Plot the ROC. If AUC > 0.85 with default hyperparams, you've got something. If it's <0.7, your shower variables

aren't telling you enough and you'd need to add cluster sub-images (next tier).
Effort: 4–8 weeks. Heavier on systematics validation than on training.

3. ML-based unfolding (OmniFold or MultiFold)

What: Replace iterative Bayesian / SVD unfolding of your p_T spectrum with a classifier-reweighted unfolding. Lets you unfold without binning and over more dimensions simultaneously.
Why thesis: Methods flourish your committee will notice. Comparing OmniFold to your IBU result as a cross-check is a strong systematic.
Risks: Real time sink for validation. Only worth it if (a) your spectrum has a feature that benefits from finer-than-bin resolution, or (b) you want to spin off a methods note.
Stack: OmniFold reference implementation on GitHub (`ericmetodiev/OmniFold`). `tensorflow` or `pytorch`.
First test (1 day): Run the demo notebook from the OmniFold repo on a toy 1D Gaussian smearing. If the closure tests pass, then port your own MC into the same format.
Effort: 8–12 weeks if you want it as a thesis cross-check, longer for a methods paper.

2 Tier 2 — LLM / agent tools that make you faster (productivity)

4. RAG agent over sPHENIX docs

What: Retrieval-augmented generation system. Chunk and embed: the sPHENIX TDR, internal analysis notes, your group's wiki, and the Fun4All / coresoftware code. Query with a chat interface that cites where it found the answer.
Why useful: You already studied TDR §7.1 manually. Multiply that by every section you'll touch over your PhD. Agent finds the page where the EMCal energy threshold defaults are set, or which class owns the GlobalVertexMap, in seconds.
Stack: `LlamaIndex` or `LangChain` for the framework. `ChromaDB` or `FAISS` for the vector store (local, free). `sentence-transformers` for embeddings (local) or OpenAI/Voyage embeddings (paid, better). Claude or GPT-4 for generation. Total cost: $5–20/month for embeddings/queries if you go the paid route, or $0 with Ollama running a local model.
First test (half day): `pip install llama-index chromadb`, point it at a folder with the TDR PDF and 5–10 analysis notes, ask "what's the default EMCal cluster energy threshold and where is it set in code?" If it cites the right TDR section, you're done with the proof-of-concept.
Effort: A weekend for a usable v1. Keep extending it as you collect more docs.

5. ROOT macro generator agent

What: CLI tool that takes a natural-language spec ("fit this histogram with Gaus+pol2, save canvas as PNG with title from filename regex") and emits a working ROOT C++ macro. Wrap Claude API or a local LLM with a prompt that includes your house style (your `BM.C` conventions, color palette, naming).
Why useful: You write a lot of plotting/fitting macros. You already have a house style. An agent that knows your conventions saves real hours over a thesis. Bonus: it can generate the LaTeX figure block too.
Stack: `anthropic` Python SDK or `openai`. Few-shot prompt with 3–5 of your existing macros as examples. Optional: wrap as a VS Code extension or a shell command `macgen "spec here"`.

First test (2 hours): Write a Python script that takes a string and your existing `BM.C` as the system prompt, calls Claude, and writes the output to a `.C` file. Try "plot pi0 mass distribution from this TFile, fit Gaus+pol1, sideband background subtraction, save canvas." If it gets 80% there and you only need to tweak, keep going.
Effort: 1–2 days for a useful tool. Iterate as you find failure modes.

6. Calibration pipeline orchestrator agent

What: Agent that drives your iterative tower-slope calibration loop: launches the job, watches the output, decides whether convergence is good enough, re-runs with adjusted parameters if not, writes a summary. Built on top of your existing `testslope.C` / `Expo_Rebinning.C`.
Why useful: Iterative calibration is exactly the kind of supervised-loop work where an agent earns its keep. You stop babysitting the convergence.
Stack: `LangChain` or `Anthropic` tool-use API. Tools: "run_macro", "read_log", "check_convergence", "adjust_threshold". The agent's loop is *plan* → *tool call* → *observe* → *decide* → *repeat*.
First test (1 day): Write the tool functions in Python (just shell wrappers around your existing macros). Give Claude tool-use access. Ask it: "run the calibration on run XYZ, iterate until per-tower slope converges to <1%, summarize."
Effort: 1–2 weeks for a robust version. The hard part is making the convergence criterion principled, not the agent plumbing.

7. Fit quality classifier (canvas PNG → good/bad)

What: Vision model that classifies your saved canvas PNGs from `BM.C` as "good fit / bad fit / weird background." Either fine-tuned ResNet on a few hundred labeled examples, or zero-shot with Claude/GPT-4V (just send the PNG and ask).
Why useful: You have $N_{\text{centrality}} \times N_{p_T}$ fits per pass, easily 100+ canvases. Manually eyeballing them is the bottleneck. A classifier flags the 10% that need attention.
Stack: Cheapest path: `anthropic` SDK with `claude-3-5-sonnet` or vision-capable model, send base64 PNG, prompt "rate this π^0 peak fit on a 1–5 scale, list any visible problems." Cost: ~$0.01 per canvas. For 100 canvases, ~$1.
First test (30 minutes): Take 10 of your existing canvas PNGs, send them to the API one by one, see if the model's quality assessments match yours. If yes, scale up.
Effort: A day for a working tool. Worth doing now because it's so cheap.

8. Systematic uncertainty bookkeeper agent

What: An agent backed by a structured spreadsheet/JSON of every systematic in your analysis. Knows which sources affect which p_T bin, propagates correlated uncertainties, regenerates the LaTeX table for your thesis chapter on demand.
Why useful: The systematics table is the single most error-prone object in a thesis. Right now you'd maintain it by hand. With an agent, you say "add a 2% energy-scale systematic correlated across all p_T bins" and the table regenerates.
Stack: A YAML or JSON file as the source of truth. Python script with `anthropic` tool-use to read/write it. Generates LaTeX with `jinja2`.
First test (half day): Define the schema (source, type, value, p_T dependence, correlation),

seed it with 3 fake systematics, write a generator that emits the LaTeX table. Plug an agent on top once the data layer works.
Effort: 3–5 days. Pays off massively at thesis-writing time.

3 Tier 3 — Exploratory / nice-to-have

9. Autoencoder for hot/dead tower detection

What: Train an autoencoder on tower-occupancy maps from good runs. Towers that reconstruct poorly are flagged as anomalous (hot, dead, drifting).
Why interesting: Quality assurance automation. Could become a service you run on every new dataset. Connects nicely to your calibration chapter.
Caveat: sPHENIX QA likely already has tooling for this; check before duplicating.
Stack: `PyTorch` convolutional autoencoder on (iη, iϕ) tower maps.
First test: Train on 50 good runs, look at reconstruction error per tower, see if known bad towers light up.

10. LLM code review agent for analysis macros

What: Run your ROOT macros through an agent that knows common ROOT pitfalls: `TFile` ownership, histogram memory leaks, `TTree SetBranchAddress` gotchas, `gDirectory` surprises.
Why useful: ROOT C++ has an unusually rich library of foot-guns. A reviewer agent catches ones you'd miss.
Stack: `anthropic` SDK, prompt with a checklist of ROOT-specific issues plus your house style.
First test (30 min): Paste one of your existing macros, ask Claude to review for memory leaks, ownership bugs, and style issues. See if the feedback is real or generic.

11. Active-learning loop for hand-labeling

What: If idea #7 takes off and you build a fit-quality classifier, use active learning to prioritize which fits a human (you) should label next — the ones the model is most uncertain about. Standard uncertainty sampling.
Why useful: Drops the labeling burden by 5–10× for a given target accuracy.
Stack: `modAL` or roll your own. Trivial to implement once #7 exists.

4 What I'd skip

Don't waste cycles on these for this thesis:

- **Generative fast-sim** (GANs/normalizing flows for shower simulation). Cool, big infrastructure ask, doesn't help your measurement.
- **ML tracking.** Not your scope; sPHENIX tracking is owned elsewhere.
- **Replacing your π^0 signal+background fits with a NN.** Won't beat a physics-motivated functional form, and you'll burn months convincing reviewers.
- **ML centrality determination for O+O.** Likely owned by a collaboration task force; not a single-student project.

- **Building an agent to "write your thesis."** Tempting; LLMs hallucinate citations and physics. Use them as editors and rubber ducks, not authors.

5 Suggested order of attack

1. **This week:** Set up RAG over sPHENIX docs (#4) and the canvas PNG fit-quality classifier (#7). Both are cheap proofs of concept and pay off immediately.
2. **Next month:** Start cluster energy regression (#1) on single-photon MC. Aim for a working notebook before you optimize.
3. **Following quarter:** Decide on #2 (merged-π^0 classifier) based on whether the high-p_T reach matters for your physics story. Start the calibration orchestrator agent (#6) once #1 is converging.
4. **Year 2+:** #3 (OmniFold) as a methods cross-check if there's bandwidth. Systematics bookkeeper (#8) when you start writing chapters.

Generated for Parker Lewis. Friday signing off.